

Buster versus GCC

Compilation throughput, generated program quality, compatibility and the next useful improvements.

Buster d4ca72fb4e97ae0a410b0d03258b00495fa45e13

Measured gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0

Capability comparator: GCC16.2 source78d4ac73dd391005b895a6148cd9831e28e1208b

Buster wins compilation of the larger tested units: 3.17x versus GCC O0 and 9.00x versus GCC O2 for the 256-function input; 2.05x and 2.30x for the existing C operation fixture. These are medians of 12 matched elapsed-time ratios, with Buster winning every round and disjoint observed timing ranges.

GCC O2 wins all four generated-program runtime tests by 1.43-6.00x and emits less .text in all six workloads. Buster has lower peak command RSS in five of six cells versus either GCC mode; GCC wins tiny compilation against O0, and most short O2 compile comparisons are weaker or inconclusive.

Overall production/compiler ecosystem winner: GCC. Buster already has useful throughput and integrated-artifact strengths, but current speed evidence is workload-specific. Use GCC for broad OS/distribution bootstrap and release-quality code while prioritizing the measured Buster gaps.

GCC wins the present breadth contest: languages, mature optimization controls, established native targets, build-system compatibility, and optional developer tooling. Buster has concrete architectural strengths: one in-process artifact pipeline, explicit bounds on ordinary optimizer work, direct LLVM bitcode and bounded Wasm/SPIR-V emission, and native Windows PDB infrastructure. Those strengths are specific capabilities; they do not establish a universal speed, memory, size, or correctness winner.

The reviewed Buster source is d4ca72fb4e97ae0a410b0d03258b00495fa45e13. The capability comparator is GCC 16.2.0, released 2026-08-07, source commit 78d4ac73dd391005b895a6148cd9831e28e1208b, tag releases/gcc-16.2.0 (tag object 1831ac03fd08e3400c16b29f21762e6b326a618d). Measured runtime/compile comparator is GCC15.2 Ubuntu, not GCC16.2. Release/tag signatures were not verified.

Native Buster -O0/-O1/-O2/-O3/-Os/-Oz/-Ofast do not currently select distinct native optimization effort: their driver mapping selects FAST, and optimization_level has no native CodegenModuleOptions consumer. Shared folding, address normalization, DCE, parameter cleanup, and SSA/local promotion already exist. FAST versus GCC O0 tests a low-effort latency baseline; FAST versus GCC O2 tests a practical quality frontier, with different optimization work.

A historical 2026-09-27 shared-cloud SQLite experiment reported Buster wall time 0.96-1.23 s versus GCC 13 O0 4.38-4.42 s: GCC/Buster 3.56-4.60x, or Buster wall time 71.92-78.28% lower. Those are cross-combinations of reported ranges, not matched confidence intervals. The Buster source was ade6ac4b6ecb21f30b61b656439bac476c145e2f, not the current pin; there were no reported GCC RSS, code-size, or generated-runtime results. It is useful prior evidence, not today's winner.

Recommendations are ranked for the compiler and OS bootstrap goal. Current diagnostics establish specific latency, memory, text and runtime deltas; the payoff of a proposed repair remains unmeasured until its own controlled experiment. Keep edit/build throughput, release-code quality and supported-operation coverage separate.

Measured results and winners

Hosted Ubuntu 26.04.1, AMD EPYC 7763 (Zen 3), trusted Buster producer Clang 21.1.8. Measured GCC is Ubuntu 15.2.0-16ubuntu1; GCC 16.2 is the separately pinned capability comparator. Diagnostic branch e19b531e03db62afd8f688ecd69d7f9f111283bf changes no production source from the Buster baseline. Compiler SHA256 eb38e2d3bdf93e5eb1134c9f6965e07d0270a0821cf9e137261b8f566e6987.

Six header-free common C inputs: tiny main, unsigned xorshift, integer-array recurrence, 256 ninline functions, double recurrence, and the repository basic_c_operations.c fixture. Independent unsigned/FP checksums and fixture self-checks passed in all 18 cells. Twelve compile triplets and twelve runtime triplets rotate all six compiler orders; 216 timed object compiles and 144 runtime executions. No failed sample was dropped; outputs were removed first, then checked for nonempty content and stable SHA256.

Both compiler tracks use GNU11, wrapv, no-strict-aliasing, unsigned-char, g0, no-PIC/no-PIE and explicit baseline x86-64 ISA. Buster uses -O0 then -fregister-allocator=fast; GCC uses O0 or O2 with generic tuning. Native Buster O strings are not GCC-equivalent optimization policies. All objects link through the same GCC driver/CRT with -no-pie, outside compile timing.

Wall time is monotonic wrapper-to-completion, including GNU time and process launch. Tables give absolute medians; winner factors use the median of each matched GCC/Buster ratio, so they need not equal ratios of displayed rounded medians. Win counts are descriptive, not significance tests or simultaneous confidence bounds. Lower milliseconds and bytes are better. Runtime includes executable startup and one printf.

Resource figures are GNU-time waited-descendant CPU and maximum child peak RSS, not simultaneous aggregate process-tree memory. CPU is rounded to 0.01 s: tiny cells are unresolved, not zero-work. The 256-function compile reports median CPU 0.01 / 0.08 / 0.24 s (Buster / GCC O0 / GCC O2); the C operation fixture reports 0.01 / 0.04 / 0.05 s. Do not infer precise CPU percentage changes from these coarse observations.

Time to object

Workload	Buster FAST ms	GCC O0 ms	GCC O2 ms	Paired point winner / consistency
One trivial function	21.10	15.82	16.73	gcc-O0: GCC 1.28x (12/12); gcc-O2: GCC 1.24x (10/12)
Integer xorshift loop	19.02	16.99	20.09	gcc-O0: GCC 1.10x (12/12); gcc-O2: inconclusive (8/12 B wins)
Integer array loop	22.66	18.08	27.20	gcc-O0: GCC 1.23x (12/12); gcc-O2: Buster 1.19x (11/12)
256 ninline functions	28.36	89.96	255.72	gcc-O0: Buster 3.17x (12/12); gcc-O2: Buster 9.00x (12/12)
Scalar double recurrence	23.13	17.86	21.59	gcc-O0: GCC 1.25x (11/12); gcc-O2: inconclusive (6/12 B wins)
Existing C operation fixture	24.75	50.42	56.52	gcc-O0: Buster 2.05x (12/12); gcc-O2: Buster 2.30x (12/12)

Generated program runtime

Workload	Buster FAST ms	GCC O0 ms	GCC O2 ms	Paired runtime winner
Integer xorshift loop	174.01	101.83	95.91	GCC O0 1.71x; O2 1.82x
Integer array loop	227.10	103.42	37.80	GCC O0 2.19x; O2 6.00x

Workload	Buster FAST ms	GCC O0 ms	GCC O2 ms	Paired runtime winner
256 ninline functions	89.51	85.20	62.77	GCC O0 1.05x; O2 1.43x
Scalar double recurrence	314.12	235.99	95.94	GCC O0 1.33x; O2 3.27x

Object code and peak memory

Workload	.text bytes B / G0 / G2	B .text vs G2	Peak MiB B / G0 / G2	B peak vs G2
One trivial function	8 / 15 / 7	+14.3%	21.71 / 18.40 / 21.36	+1.7%
Integer xorshift loop	265 / 95 / 77	+244.2%	21.81 / 22.39 / 24.30	-10.2%
Integer array loop	580 / 307 / 359	+61.6%	21.81 / 23.58 / 27.37	-20.3%
256 ninline functions	26438 / 10004 / 5939	+345.2%	23.81 / 30.88 / 36.96	-35.6%
Scalar double recurrence	236 / 112 / 83	+184.3%	21.86 / 23.04 / 25.08	-12.9%
Existing C operation fixture	17547 / 7103 / 898	+1854.0%	23.58 / 28.71 / 30.26	-22.1%

Runtime cost relative to GCC O2

Integer xorshift loop		1.82x slower
Integer array loop		6.00x slower
256 ninline functions		1.43x slower
Scalar double recurrence		3.27x slower

GCC O2 =1x. Longer bars mean slower Buster-generated programs. These microbenchmarks do not predict an application portfolio.

Correctness and driver probes

Probe	Buster observed	GCC15 observed	Conclusion / owner
Unused local + Wall/Wextra/Werror	Success; no diagnostic, both operations	Unused-variable error, both operations	GCC warning policy; evidence added to #1574
Invented positive warning flag	Success; no diagnostic	Unrecognized-option error	Silent capability promise; #1574
int a[-1];	Syntax-only succeeds; object fails with invalid-IR refusal	Both reject with negative-bound diagnostic	Known frontend defect; active owner #1568
MMD/MF, ASan, LTO, PGO, analyzer	All five positive option probes refused	All five accepted	Capability acceptance only; depfile feature #2302
Undeclared name, missing semicolon, duplicate definition, #error, missing include	Both operations reject all five	Both operations reject all five	These negative controls pass; no overall conformance score

Probe	Buster observed	GCC15 observed	Conclusion / owner
Explicit #warning	Warning and success	Warning and success	Buster has a warning producer; severity policy remains distinct

Fourteen probes run in syntax-only and object operations for both compilers: 56 untimed observations. Feature acceptance does not establish complete instrumentation or LTO/PGO behavior. For the missing semicolon, GCC explains the expected delimiter while Buster gives a broader unsupported-statement message. Raw diagnostics are retained.

Limits and excluded evidence

This is one diagnostic host, six small inputs, one Buster allocator, GCC O0/O2, and object-only compile timing. No broad application suite, large pristine SQLite, header-heavy preprocessing, C++/Rust, multi-TU scaling, full native linker latency, -g debug cost, Os/O3/LTO/PGO recipes, energy, PMU or dedicated Zen 5 acceptance was measured. There is no defensible universal aggregate speedup, conformance percentage, or total-memory winner.

Ubuntu GCC distribution hardening defaults were retained: disassembly shows ENDBR64 and an array-loop stack canary. Buster also emits stack probes, using a different strategy. Thus this is a stock-toolchain comparison with matched ISA/C/PIC policy, not matched hardening or ABI-generated prologue policy. No hardening ablation was run, and observed quality wins must not be attributed entirely to one optimization.

The wrapper floor was median 1.55 ms, with a first-sample outlier 59.17 ms; no floor was subtracted. Short compiler timings vary, and scalar-float compilation versus GCC O2 is effectively tied: paired median GCC/Buster 0.998, six wins each. There was no same-source rebuild control, A/A acceptance experiment, simultaneous interval family, or thermal/frequency isolation. Longer runtime cells have nonoverlapping observed ranges, but remain host-specific diagnostics.

First run 37004946749 failed before compiler build because a shallow checkout lacked baseline history. Second run 37005042712 passed behavior checks but its wall times were rejected for timeout-wait polling quantization. Corrected run 37005475203 is the sole current timing source. Failed and rejected attempts remain linked on #2274. No preferred-result retry or discarded failed timing cell was used.

All 18 checked programs passing does not imply the monorepo test suite, self-host fixed point, sanitizer, cross-platform, or hardware acceptance gates passed. Those were not run for this research-only branch. The built compiler is 18,702,768 bytes, but GCC driver size alone would omit cc1/runtime/tools, so no installation-footprint winner is declared.

GCC O2 reduces the C operation self-check fixture to 898 text bytes versus Buster 17,547; this is a 19.54x ratio, not representative application-size evidence. Such tests can be folded aggressively. The synthetic noline workload deliberately suppresses call inlining, so its runtime result does not measure the prize from adding an inliner.

Capability comparison

Winners below describe verified capability or support breadth. Architectural strengths are separate from measured speed. Mode names, test counts and source size do not establish conformance or performance scores.

Measured performance and evidence

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Object compile wall time	3.17x/9.00x faster on 256 functions versus G0/G2; 2.05x/2.30x on C fixture.	Wins tiny O0 and several short cells; O2 scalar-float paired tie.	Split by workload	Use paired factors and raw samples above. Large-unit Buster wins are robust within this run; no universal speedup. #2274 / #1931.
Compiler CPU and process costs	256-function median CPU 0.01s; C fixture 0.01s.	Matching O0/O2 CPU 0.08/0.24s and 0.04/0.05s.	Buster: large tested inputs	GNU-time granularity 0.01s prevents precise tiny-cell CPU ratios; full process-count census and hardware counters remain unrun. #2274.
Peak compiler memory	Lower in five of six cells versus both GCC modes; 256-function peak 23.81MiB.	256-function peak 30.88/36.96MiB; lower tiny peak.	Buster: most tested cells	Maximum waited-child peak, not simultaneous tree total. See per-cell table; no installation-footprint or whole-build memory verdict. #1931.
Generated code-section bytes	8/265/580/26438/236/17547B across six inputs.	O2 7/77/359/5939/83/898B; smaller in all six.	GCC O2: all six	Ordinary fixture folding is atypical; noinline fixture Buster 4.45x text, FP 2.84x. Attribute with independent disassembly, not container bytes. #1927 / #49.
Generated-program runtime	All four checked programs slower; runtime gaps 1.43-6.00x versus GCC O2.	O0 and O2 win every paired round of all four workloads.	GCC: all four	Source mechanisms include persistent XMM values and SSE2 loops; no per-pass recovery percentage proven. #1927 / #1944 / #49.
Tiny compilation / whole-build scaling	Tiny median 21.10ms; current integrated TU workers exist.	Tiny O0 median 15.82ms; O2 median 16.73ms.	GCC tiny; scaling unknown	Refresh current startup census; merged lazy-table work invalidates old 124M floor. Multi-TU and large/header-heavy curves remain unrun. #1295 / #53 / #424.

Frontend and existing software compatibility

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Source-language breadth	One active C frontend; removed custom language remains retired.	12 source frontend inventory entries, with experimental entries and unequal maturity.	GCC: breadth	The inventory difference is +11 frontends, not a conformance or speed percentage. Prioritize OS-required C/C++ rather than duplicating every language.

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
C dialect and standards coverage	GNU89/99/11/17/23; strict C99/11/17/23 spellings, default GNU17.	C90 through C23 plus experimental C2Y; documented substantial C99/C11 support.	GCC: documented breadth	Accepted mode names do not prove full conformance. Publish independent positive/refusal corpus results and implementation-defined choices. #423 / #309.
C++ software bootstrap	No active C++ frontend; cannot directly compile existing C++ software.	Established C++ frontend across major language generations and runtime ecosystem.	GCC: capability	Use GCC to bootstrap C++ components; evaluate a bounded canonical-IR C++ frontend against actual OS dependencies before committing to full language coverage.
Command-line contract and build queries	Useful common flags and bounded response files; several familiar options refused or absorbed.	Established compile modes, forwarding, tool queries, sysroots, and build-system conventions.	GCC: compatibility breadth	Census exact flags used by target software; unsupported semantic flags must fail visibly. Warning acceptance needs policy work, not a compatibility claim. #1574 / #2274.
Dependency-file generation	Explicitly refuses -M/-MM/-MD/-MMD/-MF/-MT/-MP; refusal fix is already landed.	Emits dependencies, selected targets, omitted system headers, and phony header rules.	GCC: capability	Implement retained include-closure emission with independent Make/Ninja and escaping controls. New owner #2302; #1232 stays closed on its intentional refusal fix.

Native optimizer and machine code

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Optimization effort and IR strategy	Typed canonical IR plus MIR; accepted native -O levels share FAST policy.	GIMPLE SSA/RTL with documented O0/O1/O2/O3/Os/Oz/Og policies.	GCC: user control	Make native effort policies truthful while retaining bounded fast mode; flags alone do not define equivalent pass sets. #47.
Cheap canonical cleanup	Production folding, address normalization, DCE, parameter cleanup, and SSA/local promotion; explicit work budgets.	Broader repeated propagation, redundancy, dead-store, and CFG optimization machinery.	GCC: breadth; Buster: explicit bounds	Measure surviving misses before adding passes. Existing cleanup must be extended and independently validated. #49 / #125.
Inlining and call overhead	No production body-substituting native inliner found in inspected paths.	Local and interprocedural inlining with profitability and code-growth controls.	GCC: capability	Begin bounded leaf/always_inline admission with recursion, linkage, debug, ABI, and growth controls; current runtime/byte prize remains unmeasured. #48.
Loop, range, and alias optimization	Conservative shared cleanup; no general production loop/alias optimizer discovered.	SSA alias/range analyses, invariant motion, induction transforms, PRE, and dead-store work.	GCC: transformation breadth	Prioritize observed recurrent loops and legal cheap idioms; preserve side effects and compile budgets. #49 / #1943 / #1949.
Automatic vectorization versus explicit SIMD	Explicit vector/SIMD lowering exists; no production scalar-to-vector loop/SLP pass found.	Loop and SLP vectorizers with legality and target cost models.	GCC: measured loop quality	Array-loop runtime 6.00x faster, with SSE2 packed disassembly; explicit SIMD is different. Separate vector and scalar/phi ablations before crediting a fix. #1944 / #49.

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
FAST allocation	Forward block scan, lazy spills, interblock contracts, frame coloring, and rematerialization.	IRA regional/global allocation and LRA target constraints, splitting, and reload machinery.	Quality/speed: unmeasured	Broader GCC machinery suggests candidates, not an automatic win. Compare spills, copies, bytes, runtime, and compiler cost. #56 / #311 / #1722.
QUALITY allocation	FAST-based bounded callee-saved pins and regional split probes; limited candidate populations.	Integrated allocation costs and register-class constraints across regions.	Unmeasured	Do not enable QUALITY by analogy. Require positive and negative workloads plus stable compiler-work bounds. #311 / #312 / #313 / #1930.
Instruction scheduling	QUALITY pressure-first deterministic DAG; static placement score, no general latency/hazard model.	Target timing-aware RTL scheduling and instruction combination before/after allocation.	GCC: model breadth	First reproduce machine-code bottlenecks; static spill reduction is not proof of runtime benefit. Keep scheduling improvements bounded. #56 / #311.
Scalar floating-point register residency	Native scalar FP recipes bridge through fixed vector registers and GPR/frame homes.	Scalar FP virtual registers remain in the target FP bank through allocation.	GCC: measured 3.27x runtime	236 vs83 text bytes; GCC keeps x/constants in XMM, Buster bridges via GPR/stack. Extend FP bank residency and phi placement; no3.27x recovery promise. #1927.
User-program profile-guided optimization	No native profile-generate/use driver contract; compiler's own Clang PGO is separate.	Profile collection/use drives branch, inline, loop, and layout decisions.	GCC: capability	Establish training/holdout workloads and stale-profile rejection before adding native PGO; do not attribute host-built compiler PGO to generated programs. #47 / #2274.
Native link-time optimization	Separate per-TU objects; no native -flto pipeline.	GIMPLE object streaming, resolution, whole-program analysis, and partitioned LTRANS.	GCC: capability	Measure cross-TU opportunities first; preserve canonical IR and independent linkage validation before a bounded LTO design. #47 / #48 / #2274.

Targets, ABI, and artifacts

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Native ISA and ABI breadth	Native x86-64/AArch64; bounded non-native targets are distinct contracts.	Broad desktop/embedded ISA families including RISC-V, ARM32, PowerPC, and AVR.	GCC: breadth	Six Buster architecture IDs are not six production contracts; GCC config directory counts are not architecture counts. Add roadmap-required targets only. #309.
Cross-target packaging	One executable selects targets without installing separately configured compiler drivers.	Typically target-configured toolchains, with some multilib facilities.	Buster: packaging design	No binary-size or deployment-time win measured. Validate each target, environment, and runtime contract independently. #309 / #423.
Native ABI interoperability	SysV/Win64/AArch64 paths; explicit compatibility boundaries and MinGW identity concern.	Established configured ABI/environment controls and runtime combinations.	GCC: environment breadth	Refuse unsupported environment aliases or implement WindowsGNU explicitly; differential aggregate/varargs/layout checks matter. #1492 / #1452 / #1757.

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Object emission and assembler integration	Own ELF64/COFF/Mach-O64 readers/writers and native encoding, with checked format boundaries.	Mature compiler plus platform assembler/object tool ecosystem.	Buster: integration; GCC toolchain: breadth	Measure complete artifact operations separately from compiler proper; independent readers must validate objects and relocations. #309 / #2274.
Linking modes and shared libraries	Built-in executable modes; native shared/PIE writer only x86-64 Linux.	External platform linkers provide extensive image modes and option support.	GCC toolchain: breadth	AArch64 ELF shared, PE DLL, and Mach-O dylib writers remain bounded expansion work. #1604; avoid silently accepting unsupported linker semantics.
Direct WebAssembly emission	Direct bounded Wasm32/Wasm64 backend contracts.	No core Wasm backend in the inspected GCC configuration.	Buster: specific capability	Keep validated subset, imports, ABI, and runtime evidence explicit; this does not imply broader language compatibility. Existing Wasm owner / #309.
Direct SPIR-V compute and offload models	Bounded direct Vulkan 1.2 SPIR-V compute emitter; external GPU orchestration is separate.	OpenMP/OpenACC offload with target toolchains; no direct SPIR-V backend inspected.	Split by contract	Buster wins direct-format capability; GCC wins established offload programming-model breadth. Validate refusals and real device behavior under existing GPU work.
Direct LLVM bitcode interchange	Binary LLVM bitcode emitter without LLVM dependency; important ABI/debug/SIMD/TLS exclusions.	GIMPLE LTO rather than direct LLVM bitcode output.	Buster: specific capability	An external LLVM consumer is a different configuration; do not claim native sanitizers, PGO, or LTO from bitcode emission. LLVM_BITCODE.md / #309.

Diagnostics, debugging, and developer tooling

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Warnings and severity policy	Wall/Wextra/Werror unused-local probe succeeds silently; arbitrary positive warning flag also accepted.	Warning groups, enable/disable rules, severity controls, and -Werror.	GCC: capability	Independent syntax/object controls posted on #1574; reuse structured producers and enforce explicit supported warning policy.
Macro/source diagnostic provenance	Invocation and original positions plus flat notes; no reconstructed expansion stack.	Macro expansion tracking and richer related-location rendering.	GCC: documented detail	Lazily expose existing retained provenance; test nested includes/macros and deterministic note ordering. Do not invent expansion frames. #1574 / #2274.
Machine-readable diagnostics	Structured C API records and terminal renderer; no documented CLI SARIF sink.	Documented SARIF and multiple diagnostic output sinks.	GCC: CLI integration	Add a minimal versioned serializer over current records; stable codes and escaping need tests. #328 is landed infrastructure; extension tracked under #2274.

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
DWARF options and consumer interoperability	Own DWARF4 writer; carries selected foreign DWARF5; compressed/split debug refused or outside boundary.	Multiple DWARF versions, split/compressed output, and variable-tracking controls.	GCC: breadth	Prioritize independent consumer correctness before format breadth; historical defects are not current reproductions. #1440 / #1452.
Lexical scope identity and variable ranges	Canonical local scope depth can synthesize full-function scopes; sibling identity concern remains.	Source lexical TREE_BLOCK identity is retained through lowering.	GCC: source model	Reproduce sibling scopes and optimized variable views with independent debuggers before a precise scope-ID/range extension. #2241.
Windows CodeView/PDB capability	Own CodeView/PDB and native Windows paths; consumer correctness work remains.	Toolchain debug formats do not imply MSVC PDB interoperability.	Buster: specialized capability	No blanket debug-quality winner without consumer tests; record exact target and debugger acceptance. #1440 / #1452.
Generated-program sanitizers	No native -fsanitize contract; sanitizing Buster itself is a separate validation activity.	Target-dependent address, undefined-behavior, thread, leak, and related instrumentation.	GCC: capability	Start with justified trap-only checks and ABI/side-effect tests if needed; runtime support and optimizer legality are explicit work. #2274.
Optional static analysis	No documented native -fanalyzer equivalent.	Interprocedural symbolic analyzer, with documented coverage limitations.	GCC: tooling capability	Keep external analyzer CI; evaluate a narrow high-value native diagnostic instead of imposing analyzer cost on ordinary compilation. #1574 / #2274.
Generated-program hardening	No stack-protector prologue; disabling spelling is accepted.	Stack protection, stack-clash checks, and target-dependent hardening options.	GCC: capability	A positive hardening feature needs canary/frame lowering, runtime ABI, negative tests, and cost measurement; no current miscompile asserted. #2274.

Build architecture, embedding, and self-hosting

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Integrated artifact pipeline	Preprocess, frontend, canonical IR, encode, object, native link, and JIT in one process.	Driver composes compiler, assembler, linker, and runtime tools.	Buster: architecture	Measure object-only and full compile-link latency independently; process integration is a hypothesis for lower overhead, not a quantified win. #2274.
Compilation parallelism	Default one worker; opt-in persistent full-TU lanes for native C link cohorts.	Usually build-system TU parallelism; LTO has parallel partitions.	Unmeasured	Establish scaling and RSS; single-TU/per-function work has separate ownership and concurrency contracts. #53 / #54 / #531 / #424.
Compiler/JIT embedding	Source/preprocessor, canonical, object, JIT lookup/release APIs exist; compact installed SDK pending.	Packaged libgccjit C/C++ + IR-construction APIs; not an arbitrary C source-string API.	GCC: packaging today	Compose existing Buster APIs with diagnostics, ownership, include overlays, and lifecycle tests. Active #1926 / #1935 / #1946.

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
Self-hosting and determinism audits	Byte-identical fixed-point checks and token/IR/MIR/diagnostic invariant audit infrastructure.	Mature native three-stage bootstrap and stage comparisons.	Both; current correctness unmeasured	Do not rank by test counts. Keep trusted-producer throughput and self-built throughput distinct; current fixed-point run was not performed here. Build guide / #2274.

Licensing and ecosystem

Aspect	Buster at pinned source	GCC capability comparator	Winner	Next step / owner
First-party redistribution clarity	No selected first-party project license grant at the pinned revision.	Explicit compiler GPL-3.0-or-later, with separately scoped component terms.	GCC: clarity	This matters for OS distribution and contributor provenance. License selection is the maintainer's decision; third-party notices do not grant Buster rights. #621.
Runtime and toolchain ecosystem	Integrated monorepo components and focused compiler APIs; deployment/compatibility census incomplete.	Established runtime libraries, target toolchains, packaged integrations, and mature downstream use.	GCC: ecosystem breadth	Quantify actual software/build success rather than a subjective maturity score; preserve per-component license and supported-operation inventories. #423 / #309 / #1926 / #621.

Prioritized improvement plan

The strongest measured generated-code priorities are scalar FP residency (3.27x runtime gap), integer-loop scalar/vector lowering (6.00x), and compact call/edge code (4.45x text on the noinline function fixture). GCC array code uses SSE2 packed operations; its FP recurrence retains x/constants in XMM registers. Buster has GPR/XMM bridges, repeated constants, stack/phi traffic and scalar address work. These are verified mechanisms, not estimates of how much any one patch will recover.

For the 256-function input, GCC O2 spends about 227 ms more compiling but saves about 27 ms per program execution: roughly nine executions repay its build cost on this host. The array loop repays GCC O2 extra compile cost on its first run. This simple compile-plus-N-times-runtime calculation is illustrative; ordinary application workloads and whole-build parallelism can change it completely.

1 - Make the comparison reproducible before promoting a performance claim

GCC's driver spawns compiler/assembler descendants, so a one-child Buster harness cannot supply equivalent CPU or RSS accounting by changing the executable name. Extend the existing operation adapter with frozen compatible inputs, explicit baseline ISA/C semantics, correctness/refusal admission, object versus compile-link operations, and trusted-producer hashes. Keep Buster FAST/QUALITY and GCC O0/O2 distinct. The current hosted GCC series above supplies bounded measurements; historical SQLite remains prior evidence. A production adapter, broad admitted corpus, full-operation series and qualified performance gates remain separate work. Completion: independently replayable raw samples, failure inventory, hashes, uncertainty, and unchanged negative controls.

Owner: #2274 / #1931 / #423

2 - Introduce truthful fast and release-quality native policies, starting with bounded inlining

GCC selects distinct GIMPLE/RTL policies and budgets local/IPA inlining. Buster's native -O spellings currently share FAST; cleanup already exists. Measure calls, code bytes, and runtime before implementing a small leaf/always_inline subset with explicit growth/work limits, recursion refusal, linkage semantics, debug provenance, and ABI controls. The expected speed/size payoff is unknown until measured per workload. Completion: positive inline cases, no-inline/recursive/linkage negatives, unchanged fast-mode contract, and compiler/runtime economics.

Owner: #47 / #48 / #49

3 - Keep scalar floating-point values in the FP register bank

Current double recurrence is 3.27x slower and 2.84x text versus GCC O2. Disassembly shows Buster GPR/XMM bridges and loop-phi stack traffic, while GCC retains x/constants in XMM. Extend existing machine value types and allocation to FP bank residency; test NaN/signed zero, call clobbers, edge copies, ABI and debug ranges across supported targets. Keep compile-work bounds and measure each change; the full observed gap is not one pass's promised payoff.

Owner: #1927

4 - Add depfiles to unlock existing C build systems

GCC emits dependency edges, requested targets, system-header filtering, and optional phony rules. Buster now explicitly refuses these flags; #1232's refusal fix is not a depfile implementation. Start with -MMD/-MF/-MT from retained include edges, deterministic escaping/deduplication and atomic writes, then a bounded -MP extension. No compile-speed payoff is claimed; quantify newly portable builds and overhead. Completion: make/ninja incremental rebuild checks for spaces, nested includes, removed headers, repeated targets, and failed compilation.

Owner: #2302; #1232 remains closed on intentional refusal

5 - Make warning flags and severity an end-to-end contract

GCC connects warning groups and -Werror to producer diagnostics. Buster accepts arbitrary -W values without equivalent policy state. Reuse existing structured records and add a bounded warning catalogue, severity state, unsupported-option behavior, and overflow/unused/prototype cases supported by current semantics. Measure accepted/rejected warning cases; no quality percentage can be inferred from GCC's warning count. Completion: each promised flag changes the expected diagnostic/exit result and unknown flags have deterministic explicit behavior.

Owner: #1574

6 - Resolve first-party licensing and derived-material provenance

GCC has an explicit compiler grant and notice-scoped runtime exception; Buster's root notices do not select a first-party license, and Arm-derived material remains unresolved. Inventory contributor/source provenance at pinned revisions and give the maintainer a concrete component-scoped decision. This is a distribution-readiness gap with no performance percentage. Completion: an explicit first-party decision plus verified per-component notices and resolution/removal/authorization of restricted derived outputs. This review does not select a license.

Owner: #621

7 - Admit high-value scalar and vector transforms from real missed-opportunity census

GCC combines SSA analyses, loop transforms, and target cost models; Buster should extend its existing bounded cleanup. Census constant div/rem, count idioms, redundant edge copies/address forms, and legal invariant motion before attempting SLP. Use current generated code and independent semantics, including overflow, shifts, aliases, volatile/atomic accesses, and traps. Amount pending matched workload evidence. Completion: targeted positive and negative controls, actual runtime/byte prize, and retained compile-work budgets; a research census is not a shipped vectorizer.

Owner: #49 / #1943 / #1937 / #1938 / #1949 / #1944 / #1928 / #1929

8 - Improve allocation and scheduling only where current quality evidence warrants it

GCC's IRA/LRA account for regional costs, classes, constraints, and splitting; RTL scheduling uses target timing. Buster already has spills, frame coloring, rematerialization, bounded pins, and splits. Begin with pressure-reducing instruction selection, consistent costs, and a reproduced spill/copy bottleneck. QUALITY and static scheduling scores are hypotheses, not global improvements. Amount pending cells. Completion: byte/runtime/compile/RSS economics across positive and negative workloads, deterministic output, ABI and edge correctness, and bounded candidate populations.

Owner: #56 / #311 / #312 / #313 / #1722 / #1930 / #125

9 - Validate debugger behavior and source scopes before widening debug formats

GCC preserves source lexical blocks and offers broader DWARF/variable-tracking policies. Buster's DWARF4/PDB emitters already contain several fixes, so old defect lists are not fresh evidence. Reproduce same-depth sibling scopes and variable lifetime/range behavior using independent consumers; add explicit scope identities/ranges where warranted. Separately serialize existing diagnostic records rather than duplicate producers. Amount is current failing consumer cases, not a guessed debug-quality score. Completion: pinned GDB/LLDB/Windows consumer evidence and compression/split refusals remain explicit.

Owner: #2241 / #1440 / #1452; diagnostic serialization extension under #2274

10 - Expand only the ABI and shared-image contracts needed by the OS roadmap

GCC's configured target/runtime ecosystem and platform linker handle more environment and image combinations. Buster should first refuse MinGW/MSVC ambiguity, then complete one necessary image writer at a time: AArch64 ELF shared, PE DLL, or Mach-O dylib. Preserve independent aggregate/varargs/layout and relocation validation. No architecture-count speedup or target-support percentage is meaningful. Completion: explicit triples, linked external C interop, loader/consumer checks, negative relocation/image controls, and no silent semantic flag acceptance.

Owner: #1492 / #1452 / #1757 / #1604 / #309

11 - Package the existing source/JIT pieces and measure practical deployment

GCC already packages libgccjit as an IR-construction API; Buster's source-string compilation, diagnostics, canonical/object components, and JIT lifecycle can become a differentiated SDK. Reuse active work instead of creating a competing façade. Bound include overlays, diagnostics ownership, import resolution, and release semantics; keep ordinary source compilation and hot reload separate operations. Size/startup/deployment advantage is unmeasured. Completion: small independently built C client, no filesystem requirement for in-memory cases, failure cleanup, import/lifetime tests, and documented target/licensing boundaries.

Owner: #1926 / #1935 / #1946

Licenses and provenance

Project or component	Verified terms	Scope / unresolved point
Buster first-party code	Missing/unselected first-party project grant	Pinned README, LICENSES/README.md, and notices do not license the complete project. Maintainer-owned #621.
Buster retained third-party material	Component scoped: XED Apache-2.0; LLVM Apache-2.0 WITH LLVM-exception; Zig MIT; xxHash BSD-2-Clause; XCB source-header terms	These notices do not grant rights to unrelated Buster code. Inventory exact files/revisions; do not treat the monorepo as uniformly licensed.
Buster Arm-derived outputs	Restricted/permission status unresolved	Retained notices require provenance and permission review under #621; no unrestricted redistribution assertion.
GCC compiler proper and libgccjit interface	GPL-3.0-or-later	Verified notices in gcc/gcc.cc, gcc/config.gcc, and gcc/jit/libgccjit.h. libgccjit header has no Runtime Library Exception notice.
GCC libgcc, libgomp, selected libstdc++ headers	GPL-3.0-or-later WITH GCC-exception-3.1 where bearing the notice	COPYING.RUNTIME exception is component/notice scoped. Representative files contain additional legacy notices; do not apply it to the compiler wholesale.
GCC libsanitizer representative files	Apache-2.0 WITH LLVM-exception; separately noticed files may differ	Representative sanitizer/ASan headers carry LLVM terms; legacy LICENSE.TXT includes NCSA/MIT. Preserve the per-file inventory.
GCC libquadmath	Separate GNU library copyleft terms	quadmath.h says Library GPL 2-or-later; COPYING.LIB is LGPL 2.1. Keep exact notices rather than infer the compiler runtime exception.
GCC manual	GFDL-1.3-or-later with stated invariant/front/back-cover sections	Documentation grant is distinct from compiler/runtime code terms; report uses concise paraphrases.

Unmeasured questions

This is a pinned-source capability review plus separately inserted hosted diagnostic measurements, not exhaustive language conformance, target support certification, production correctness proof, or physical performance acceptance.

Capability statements use GCC 16.2.0 documentation/source. Measured GCC version must be taken from the actual runner metadata and may be older. The release tag was resolved, not cryptographically authenticated.

No benchmark, build, debugger session, current bootstrap, laptop execution, or 9700X access was performed by the synthesis agent. Unrun checks are pending. Root supplies the actual diagnostic run and validation status.

Hosted CPU, timing variance, wrapper overhead, GNU time resolution, and incomplete machine/process metadata can limit small-cell conclusions. Peak RSS in the diagnostic is maximum waited-child peak, not aggregate concurrent process-tree RSS.

Object-only compilation and a common external link isolate generated-code quality; they do not measure Buster's integrated linker advantage. Compile-plus-link, debugger output, shared-image support, and deployment need separate operation-specific evidence.

No unsupported compilation may be scored as a fast win. Preserve rejected corpus cases and distinguish compile success, correct execution, supported semantics, and admitted performance cells.

Explicit baseline ISA, char signedness, overflow/aliasing rules, debug policy, PIC/PIE policy, allocator ordering, input/include hashes, and runtime harness must agree. Default -march behavior differs, and matching -O strings is insufficient.

A narrow synthetic corpus diagnoses mechanisms; it does not establish the OS/software bootstrap corpus, headers, generated-source work, ABI coverage, or full build success. No whole-monorepo speedup follows from its average.

Historical measurements were from a modified SQLite extraction and an older Buster/GCC configuration. Cross-combined endpoint ratios are not confidence intervals, and trusted-host versus self-built Buster throughput must remain separate.

Compiler IDs, configuration directories, source languages, diagnostic counts, and test counts are inventories rather than standards-conformance percentages or global quality scores.

Direct LLVM bitcode, Wasm, SPIR-V, PDB, and integrated linking are bounded contracts. Availability of an emitter does not imply full source semantics, external runtime/consumer compatibility, or native sanitizer/PGO/LTO support.

Existing issue ownership was inspected while the comparison was prepared and can change. Historical defects and closed refusal issues should not be reopened without a new current reproduction; extend owners instead of duplicating broad epics.

First-party Buster rights are unresolved, component licenses are distinct, and Arm-derived restrictions need maintainer action. No legal/license selection or blanket runtime-exception claim is made.

Published GitHub records

#2274 - full comparison, exact revisions and cloud experiment history
<https://github.com/buster14a/buster/issues/2274>

#2302 - bounded dependency-file generation feature
<https://github.com/buster14a/buster/issues/2302>

#1568 - independent negative-array-bound reproduction; existing active owner
<https://github.com/buster14a/buster/issues/1568#issuecomment-5952134740>

#1574 - warning-policy and unknown-warning controls
<https://github.com/buster14a/buster/issues/1574#issuecomment-5952135183>

#1927 / #1944 / #1295 / #47 - measured FP, vector, startup and policy evidence
<https://github.com/buster14a/buster/issues/2274>

Evidence and primary sources

Run: <https://github.com/buster14a/buster/actions/runs/37005475203>

Artifact SHA256: 47c04b2c3f12e26c1346c4dbde0face10643dbc7a0256e364495a5686fec94ec

Production tree: e3a17ff788cbfc5f4038f3c260a121315ddf561c

Buster pinned repository and implementation rules

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/AGENTS.md>

Buster compiler project and capability boundaries

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/projects/compiler.md>

Buster driver flag and optimization mappings

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/src/buster/lib/compiler/driver/driver.c>

Buster codegen configuration spellings

https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/src/buster/lib/compiler/driver/codegen_configurations.h

Buster canonical FAST pipeline

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/canonical-fast-pipeline.md>

Buster middle-end pass map

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/middle-end-pass-map.md>

Buster FAST allocation

https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/src/buster/lib/compiler/codegen/register_allocator_fast.c

Buster QUALITY allocation

https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/src/buster/lib/compiler/codegen/register_allocator_quality.c

Buster scheduling

https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/src/buster/lib/compiler/codegen/machine_schedule.c

Buster native codegen

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/src/buster/lib/compiler/codegen/codegen.c>

Buster driver, linker and debug support contract

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/agents/driver.md>

Buster diagnostics model

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/diagnostics.md>

Buster lexical debug scopes

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/src/buster/lib/compiler/debug/debug.c>

Buster LLVM bitcode subset

https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/LLVM_BITCODE.md

Buster Wasm64 contract

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/WASM64.md>

Buster parallelism contract

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/agents/parallelism.md>

Buster bootstrap/build contract

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/agents/build.md>

Buster benchmarking contract

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/agents/benchmarking.md>

Buster throughput operation and RSS scope

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/tools/throughput/throughput.c>

Historical 2026-09-27 SQLite diagnostic

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/docs/performance-audits/2026-09-27T155317Z.md>

Buster first-party license status

<https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/LICENSES/README.md>

Buster third-party component notices

https://github.com/buster14a/buster/blob/d4ca72fb4e97ae0a410b0d03258b00495fa45e13/THIRD_PARTY_NOTICES.md

GCC 16 release series

<https://gcc.gnu.org/gcc-16/>

GCC release history
<https://gcc.gnu.org/releases.html>

GCC 16.2 source pin
<https://github.com/gcc-mirror/gcc/tree/78d4ac73dd391005b895a6148cd9831e28e1208b>

GCC 16.2 language standards
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Standards.html>

GCC 16.2 optimization policies
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Optimize-Options.html>

GCC SSA optimizer mechanisms
<https://gcc.gnu.org/onlinedocs/gccint/Tree-SSA-passes.html>

GCC RTL optimizer mechanisms
<https://gcc.gnu.org/onlinedocs/gccint/RTL-passes.html>

GCC 16.2 preprocessor/dependency options
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Preprocessor-Options.html>

GCC 16.2 warning controls
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Warning-Options.html>

GCC 16.2 diagnostic sinks
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Diagnostic-Message-Formatting-Options.html>

GCC 16.2 debug options
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Debugging-Options.html>

GCC 16.2 instrumentation/hardening
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Instrumentation-Options.html>

GCC 16.2 optional analyzer
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Static-Analyzer-Options.html>

GCC 16.2 target-specific options
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Target-Specific-Options.html>

GCC 16.2 OpenMP/OpenACC
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/OpenMP-and-OpenACC-Options.html>

GCC 16.2 libgccjit contract
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/jit/>

GCC compiler license notice
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/gcc/gcc.cc>

GCC libgccjit interface license
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/gcc/jit/libgccjit.h>

GCC Runtime Library Exception
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/COPYING.RUNTIME>

GCC libgcc representative license
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/libgcc/libgcc2.c>

GCC libgomp representative license
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/libgomp/libgomp.h>

GCC libstdc++ representative notices
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/libstdc++-v3/include/std/vector>

GCC libsanitizer representative LLVM terms
https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/libsanitizer/sanitizer_common/sanitizer_common.h

GCC libsanitizer legacy license inventory
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/libsanitizer/LICENSE.TXT>

GCC libquadmath representative notice
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/libquadmath/quadmath.h>

GCC libquadmath separate library license
<https://github.com/gcc-mirror/gcc/blob/78d4ac73dd391005b895a6148cd9831e28e1208b/COPYING.LIB>

Measured GCC15.2 upstream source tag; Ubuntu package patch provenance is separate
<https://github.com/gcc-mirror/gcc/tree/5115c7e447fc07457443df874bf57840e8316d5f>